

Spring JDBC:

```
public Student findById(Long id) {

    String sql = ""
        SELECT id, name, email
        FROM students
        WHERE id = ?
        "";

    return jdbcTemplate.queryForObject(
        sql,
        studentRowMapper,
        id
    );
}
```

Hibernate

```
public Student findById(Long id) {

    return entityManager.find(Student.class, id);
}
```

BoilerPlate Methods:

```
@Repository
public class StudentRepository {

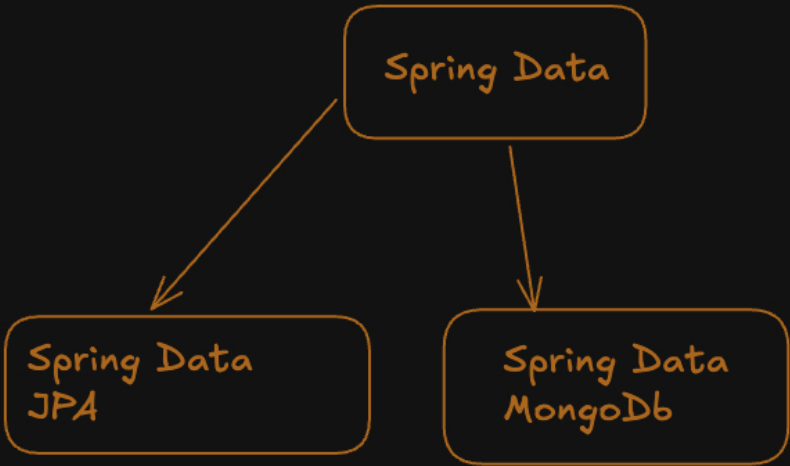
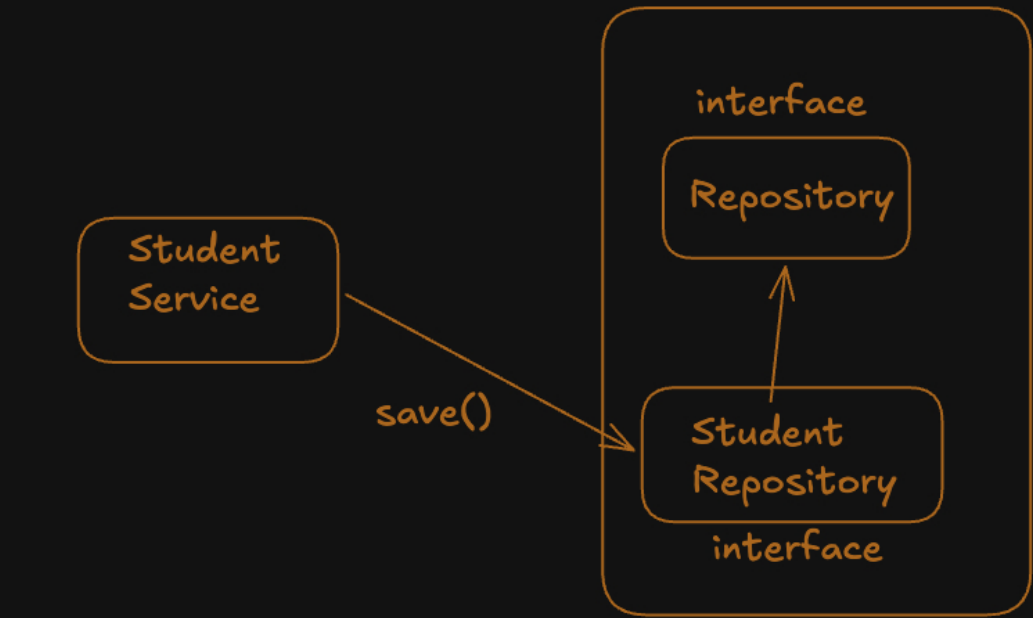
    @PersistenceContext
    private EntityManager entityManager;

    public void save(Student student) {
        entityManager.persist(student);
    }

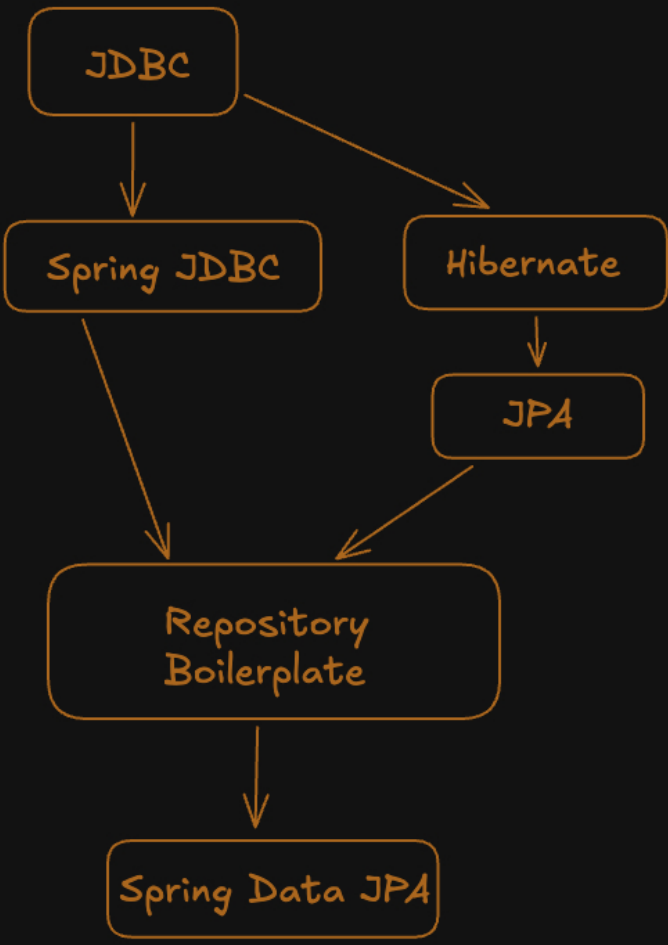
    public Student findById(Long id) {
        return entityManager.find(
            Student.class,
            id
        );
    }

    public void delete(Student student) {
        entityManager.remove(student);
    }
}
```

StudentRepository
DepartmentRepository
UserRepository



@Entity
@Column



Your repository interface



Spring Data JPA creates a runtime implementation



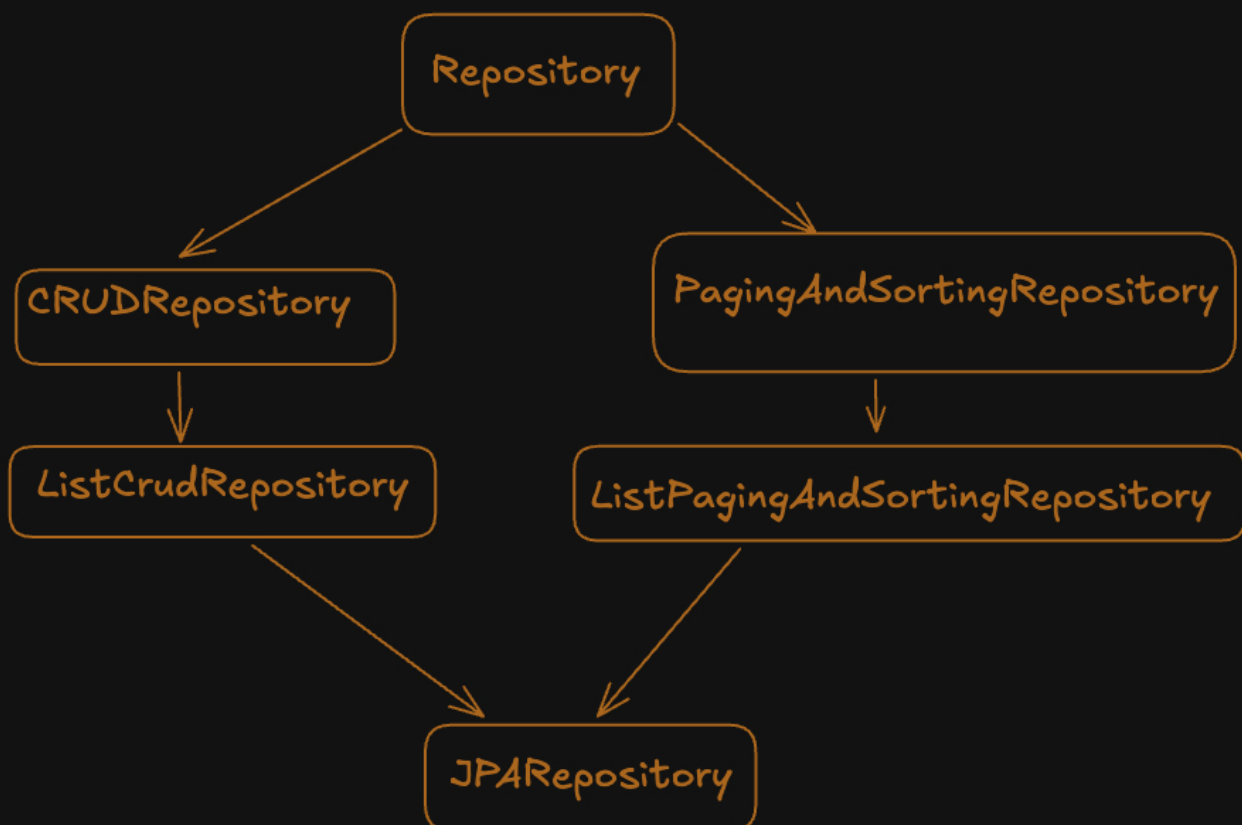
Implementation uses EntityManager



EntityManager delegates to Hibernate

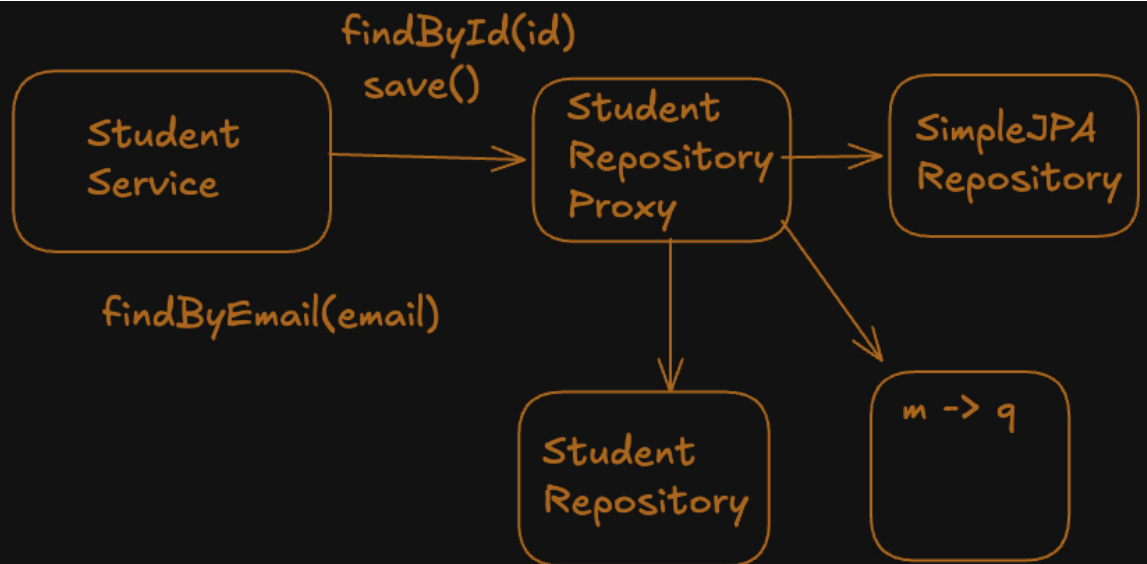


Hibernate generates SQL



Spring Data JPA Responsibilities :

1. Providing common CRUD Operations
2. Creating Repository Implementations
3. Deriving Queries from method names.
4. Executing Custom Queries
5. Supports Pagination & Sorting



class SimpleJpaRepository implements JpaRepository<Student, Long> {

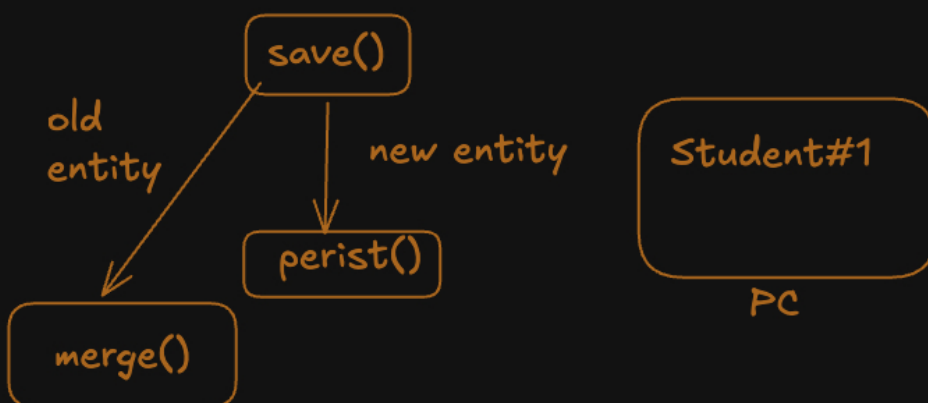
```

@Override
T save(T entity) {
    entityManager.persist(entity);
}
  
```

```

@Override
Optional<T> findById(R id) {
    return Optional.of(
        entityManager.find(T.class, id));
}
  
```

}



saveAll(List<Student> students)

saveAll()

```
for(S : students) {  
    entityManager.save(s);  
}
```

findById(id)

findAll()

existsById(id) SELECT count(*) from students
where id = ?

count()
delete(student)
deleteById(id);
deleteAll()

save();
flush();

Student

find By Email And age

↓
subject

↓
predicate

find
exists
count
delete
findTop3
findDistinct

Email/age/name
Like
greaterThan
containing
startsWith
endsWith
IN/NotIn/Null/NotNull

select * from student
where email = ? AND
age = . ?

```
select * from student  
where email like = "%tya@gmail%"
```

Order BY

```
findByEmailOrderByNameAsc();
```



```
select * from student  
where email = ?  
ORDER BY name ASC;
```

```
findTop10ByNameContainingIgnoreCaseAnd  
AgeGreaterThanAndActiveTrueAnd  
DepartmentNameOrderByAdmissionDateDesc(...)
```

JPQL (Jakarta Persistence Query Language)

```
Select * from student  
where email = :email
```

SQL

```
select s from Student s  
where s.email = :email
```

JPQL

JOINS



Explicit Joins

Student

id
name
department



Department

id
name
active

```
select s from Student s  
JOIN s.department d  
WHERE d.active = true
```

Implicit Join

```
select s from student s  
where s.department.active = true
```

```
select * from student s  
JOIN Department d  
Where s.dept_id = d.id;
```

Sort

```
ORDER BY email DESC;
```

```
Sort sort = Sort.by()
```

Pagination

page Number
page size

2,000,000

(0, 2). -> 1,2
(1, 2). -> 3, 4

Pageable

4 * 10 = 40

```
pageRequest.of( 4, 10);
```

Page

```
Page<Student> page =  
    studentRepository.findByActiveTrue(  
        pageable  
    );
```

```
page.getContent();  
page.getNumber();  
page.getSize();  
page.getNumberElements();  
page.getTotalElements();  
page.getTotalPages();  
page.hasNext();  
page.hasPrevious();  
page.isFirst();  
page.isLast();  
page.getTotalElements();  
page.getTotalPages();
```

